Project Report

Angelina Podoļako s1125886

Information Modelling: From Theory to Practice

Table of Contents

Introduction

This report is written for a pupil from the highest class in a secondary school with a science profile, who is considering enrolling in the Computer Science program at university. Such a person already has a general interest in science and technologies, but without background knowledge of Information Modelling and Databases.

In this report, I will explore and practice the foundations of Information Modelling. Information Modelling is the method that represents data in an organised and structured way, and connects it within real-world systems, such as hospitals, schools, or companies. By defining the types of objects and their relationships, information models simplify complex problems, and that provides a better understanding of how data is stored, retrieved, and related, which is essential in Computer Science and Database design.

The main goal of these assignments is to improve Information Modelling skills by practice on weekly exercises. Besides, to improve Academic Writing skills by learning how to explain complex technical things in a clear way. Throughout the weekly assignments, I progressively explored different aspects of Information Modelling. Starting with simple tables representing patients, I learned to structure data systematically and define relationships between objects. Gradually, I applied Object Role Modelling (ORM) methods, including Object Role Normal Form (ORNF) and Object Role Grammar Rules (ORGR), to formalize information about different Universes of Discourse, such as companies, childcare organizations, athletes, and students. I practiced specifying integrity constraints to ensure data correctness and completeness and used Object Role Calculus (ORC) to navigate information structures and answer complex queries.

Additionally, I studied specialization and generalization concepts, learning to create hierarchies of object types that reflect real-world relationships while avoiding data duplication. A final step of this project was a model transformation to move forward from Part A to Part B of the Information Modelling and Databases.

The following sections present the results of each assignment, including tables, diagrams, and explanations of the applied methods, demonstrating the development of my Information Modelling skills over time.

Assignment 1

A Small Example about a Hospital

No.	Full Name	Date of birth (dd/mm/yyyy)	Phone Number
1	Nataly Salnikova	17/04/2001	12345678
2	Elena Smart	05/08/1982	23456789
3	Eveline Robins	01/01/2001	34567891
4	Mary Jane	18/08/1993	45678912

Table 1. A small example about a hospital.

In this first assignment, I created a small and simple table (see Table 1 above) to model an example of information that could be stored in hospitals about patients. The table above consists of four columns, describing each patient's features*: No., Full Name, Date of birth and Phone Number*. Each row represents one object, in this example a patient, describing how features are related to each object.

This model does not cover all possible details of patients, for example, Doctor Name, City, Department, etc. However, this simplified example shows the basis of Information Modelling, and how information about real-world situations can be stored in a structured way.

Assignment 2

First Information Structure

In this assignment, I was given two sample tables (see Table 2 and 3 below) about a company, their customers, and the salespersons who visit them. The main goal of the assignment is to transform these tables into a resulting information structure by applying three steps:

- Step A: Give sentences in Object-Role Normal Form (ORNF).
- Step B: Give the corresponding Object-Role Grammar Rules (ORGR's).
- Step C: Give the resulting information structure (Object-Relational Mapping, ORM).

Information structures help us to organise knowledge about the real world so that a computer can store and process it without errors and ambiguity. They are built on the basis of a universe of Discourse (UoD), it defines the specific domain of a real world that we study (for example, in this assignment it is the company, their customers and salespersons). Through the three steps defined above (ORNF → ORGR → resulting Information Structure), the usual sentences about the UoD are gradually transformed into a formal structure. This formal structure forms the foundation for an Information Model, which can be later implemented in a database system.

CustomerAssignment:

CustID	CustName	City	Salesman
K1	de Vries BV	Nijmegen	Jansen
K2	Staal NV	Nijmegen	Peters
K3	de Vries BV	Arnhem	Jansen

Table 2. CustomerAssignment.

CustomerStatus:

Salesman	Date of visit	CustID	Impression	Date next visit
Peters	15-10-2005	K2	rather reserved	25-11-2005
Jansen	16-11-2005 16-11-2005	K1 K3	positive encouraging	15-12-2005
Peters	25-11-2005	K2		

Table 3.CustomerStatus.

Step A: ORNF sentences.

In this step, we write the facts about real world as a normal, simple sentences, in structured way to avoid confusions. That is done to have well-formed sentences that are ready for information modelling. The general procedure to remove syntactic variation:

1. First split sentences into elementary sentences.
2. Then group the resulting sentences by their deep sentence structure.
3. Make constants (such as names and numbers) fully qualified.
4. Use standard names for all entities.
5. Add layout conventions to avoid ambiguity.

From the tables CustomerAssignment and CustomerStatus, I derived the following independent facts.

Customer facts (CustomerAssignment table):

1. Each **Customer** has a **CustID**.
2. Each **Customer** has a **CustName**.
3. Each **Customer** is located in a **City**.
4. Each **Customer** is assigned to a **Salesman**.

i. Customer with CustID ‘K1’ has CustName ‘de Vries BV’.
ii. Customer with CustID ‘K1’ is located in City ‘Nijmegen’.
iii. Customer with CustID ‘K1’ is assigned to Salesman with Name ‘Jansen’.
iv. Customer with CustID ‘K2’ has CustName ‘Staal NV’.
v. Customer with CustID ‘K2’ is located in City ‘Nijmegen’.
vi. Customer with CustID ‘K2’ is assigned to Salesman with Name ‘Peters’.
vii. Customer with CustID ‘K3’ has CustName ‘de Vries BV’.
viii. Customer with CustID ‘K3’ is located in City ‘Arnhem’.
ix. Customer with CustID ‘K3’ is assigned to Salesman with Name ‘Jansen’.

Visit facts (CustomerStatus table):

5. Each **Salesman** visits a **Customer** on a **VisitDate**.
6. Each **Visit** is carried out by a **Salesman**.
7. Each **Visit** concerns a **Customer**.
8. Each **Visit** may have an **Impression**.
9. Each **Visit** may have a **NextVisitDate**.

x. Salesman with Name ‘Peters’ carried out Visit on 15-10-2005 concerning Customer with CustID ‘K2’.
xi. Visit on 15-10-2005 has Impression ‘rather reserved’.
xii. Visit on 15-10-2005 has NextVisitDate 25-11-2005.

xiii. Salesman with Name ‘Jansen’ carried out Visit on 16-11-2005 concerning Customer with CustID ‘K1’.
xiv. Visit on 16-11-2005 has Impression ‘positive’.
xv. Visit on 16-11-2005 has NextVisitDate 15-12-2005.
xvi. Salesman with Name ‘Jansen’ carried out Visit on 16-11-2005 concerning Customer with CustID ‘K3’.
xvii. Visit on 16-11-2005 has Impression ‘encouraging’.
xviii. Visit on 16-11-2005 has no NextVisitDate.
xix. Salesman with Name ‘Peters’ carried out Visit on 25-11-2005 concerning Customer with CustID ‘K2’.

Step B: ORGR.

Next, we take sentences from step A and prepare them for formalization. First, qualify labels – make all object labels unique and fully descriptive, add standard names – ensure that the same type of object always uses the same name, and then add layout conventions – place roles on separate lines, use consistent brackets and spacing. After that, we make an abstraction of this sentences by 1) removing instances, 2) writing standard names between brackets. This results in ORGR, which is an abstraction of a sentence in ORNF. These rules show the relationships of objects, without writing them as long sentences.

Customer ORGR (CustomerAssignment table):

1.Customer (CustID) has (CustName).
2.Customer (CustID) is located in (City).
3.Customer (CustID) is assigned to Salesman (SalesmanName).

Visit ORGR (CustomerStatus table):

4. Salesman (SalesmanName) carries out Visit (VisitDate) concerning Customer (CustID).
5.Visit (VisitDate) has (Impressiontext).
6. Visit (VisitDate) has (NextVisitDate)

Remark:
Customer (CustID) is located in (City): we write it this way when City is just an attribute with no unique identity or additional properties.
Customer (CustID) is located in City (CityName): we write it this way when City is treated as a separate object that can have its own properties or participate in other relationships.

Step C: Resulting Information Structure (ORM Scheme).

Finally, we create a diagram (see diagram 1 below) that shows the objects and how they are connected with each other.

In the diagram:

- Circles: represent abstract object types or label types.
- Rectangles : indicate fact types (relationships) connecting the objects.
- Bridge types: often appear as small connectors or inside circles to link abstract objects with their labels.
- Binary relationships connect two objects (two rectangles), ternary connect three (three rectangles).

ORGR contains the structure of your information:

- Object types
  - Abstract object types (like standard names)
  - Label types (like concrete names or numbers)
- Fact types (relationships connecting objects)
- Bridge types (special relations connecting abstract objects with labels).

Relationships can be binary (between two objects) or ternary (involving three objects), and that depends on the complexity of the sentence. Also, it is important to mention, that bridge types are sometimes “hidden” in diagrams for simplicity – they are represented inside circles or small connectors so the diagram stays clean (shorthand notation).

These diagrams, or also parse trees, visualizes the information structure from step B, and further it can be used as a basis for database implementation.

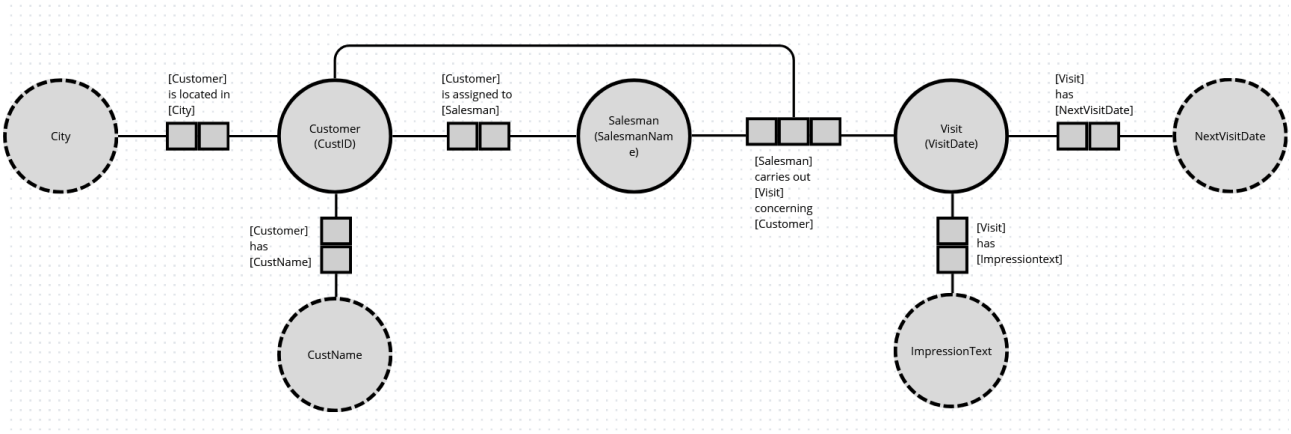


Diagram 1. Resulting Information Structure.

Assignment 3

First Integrity Constraints

In the third assignment, I worked with a new table (see Table 4 below) that describes a childcare organization and its locations. The goal of this week’s assignment is to specify my first integrity constraints. The work was done in five steps, where first three are already known from previous assignment:

- Step A: Give sentences in ORNF.
- Step B: Give the corresponding ORGR's.
- Step C: Give the resulting information structure.
- Step D: Add the integrity constraints to the information structure.
- Step E: Place the original data instances (the population of the original table) inside the types of the information structure and check that your constraints are correct.

In the design of information models, we already know three things: UoD – real world, Conceptual Schema – formally describes that world, its objects and their relations , and State of UoD – uniquely characterized by facts that holds at the moment. State of UoD is the Population of Schema – that means exactly these facts, that is, specific instances of data that we put inside the schema to show the current state of the domain. In table 4 example, at UoD we are considering a children's organization. In the schema, we have instances *Location* and *Cooperator*. And the population is a table where

"Speel Goed", "De Oppas" are indicated in *Location*, and in *Cooperator* "M1", "M2" and the connections between them. Thus, the schema sets the rules, and the population (state) shows the specific data corresponding to these rules.

Integrity constraints are needed in order for population to be a valid state for this schema. In other words, constraints are the same as rules for population, which specify that all data are correct and not contradictory. Two types of constraints:

1. A uniqueness constraint makes sure something cannot repeat when it shouldn't. Example: each *Location* must have its own unique name. In the schema, it is indicated as associating arrow symbol with role.
2. A total role constraint ensures that every object plays a role in the model. Example: every *Cooperator* must have a M-number. In the schema, it is indicated by associating dot with role.

When these constraints are added, the schema becomes not only a structure of objects and relationships, but also a semantic conceptual schema: it defines which population is or impossible.

ChildcareCentra:

Location	City	Cooperator	Telnr	Appointed for (hrs/week)	Speciality
Speel Goed	Nijmegen	M1	0644332200	20	first aid
De Oppas	Arnhem	M2	0644332211	20	-
		M3	0655443322	10	Administration
De Paddestoel	Nijmegen	M3	0655443322	15	ADHD

Table 4. ChildcareCentra.

Step A: ORNF sentences.

1. **Cooperator** with Name "M1" works in **Location** with Name "Speel Goed".
2. **Cooperator** with Name "M2" works in **Location** with Name "De Oppas".
3. **Cooperator** with Name "M3" works in **Location** with Name "De Oppas".
4. **Cooperator** with Name "M3" works in **Location** with Name "De Paddestoel".
5. **Location** with Name "Speel Goed" is in **City** with Name "Nijmegen".
6. **Location** with Name "De Oppas" is in **City** with Name "Arnhem".
7. **Location** with Name "De Paddestoel" is in **City** with Name "Nijmegen".
8. **Cooperator** with Name "M1" has **Telnr** "0644332200".
9. **Cooperator** with Name "M2" has **Telnr** "0644332211".
10. **Cooperator** with Name "M3" has **Telnr** "0655443322".
11. **Cooperator** with Name "M1" has **WeeklyHours** "20" (hrs/week) in **Location** with Name "Speel Goed".
12. **Cooperator** with Name "M2" has **WeeklyHours** "20" (hrs/week) in **Location** with Name "De Oppas".
13. **Cooperator** with Name "M3" has **WeeklyHours** "10" (hrs/week) in **Location** with Name "De Oppas".
14. **Cooperator** with Name "M3" has **WeeklyHours** "15" (hrs/week) in **Location** with Name "De Paddestoel".
15. **Cooperator** with Name "M1" has **Speciality** "first aid" in **Location** with Name "Speel Goed".
16. **Cooperator** with Name "M3" has **Speciality** "Administration" in **Location** with Name "De Oppas".
17. **Cooperator** with Name "M3" has **Speciality** "ADHD" in **Location** with Name "De Paddestoel".

Step B: ORGR.

Objectification in ORM is the process of turning a fact into a separate object type. For example, instead of just stating that a cooperator works certain hours in a location, we can create a new object called Worker. This object can then have its own attributes, like weekly hours or speciality, and allows additional facts to be attached to it

1. Worker: **Cooperator** (Name) in **Location** (Name).
2. **Location** (Name) is in **City** (Name).
3. **Cooperator** (Name) has **Telnr** (Number).
4. Worker(**Cooperator, Location**) has **WeeklyHours** (hrs/week).
5. Worker(**Cooperator, Location**) has **Speciality** (Description).

Step C: Resulting Information Structure (ORM Schema).

Resulting Information Structure.

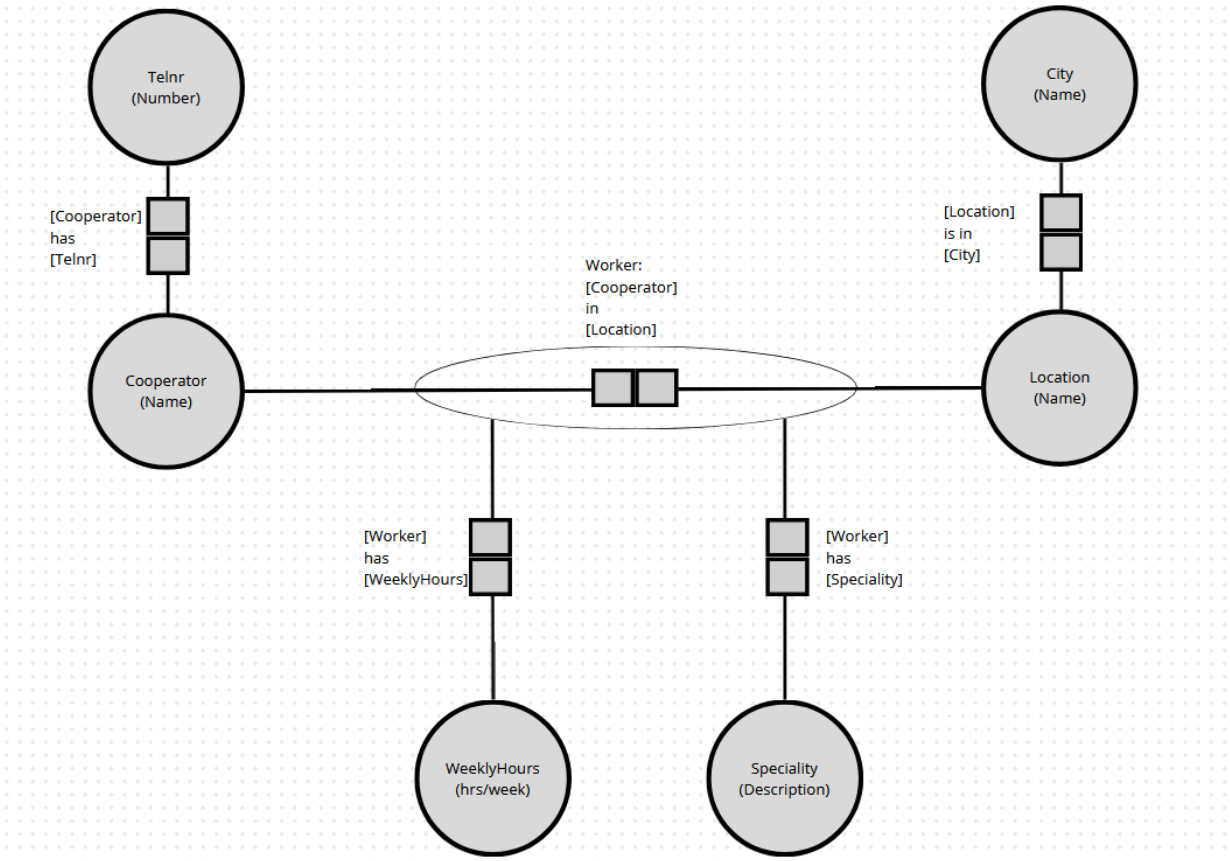


Diagram 2. Resulting Information Structure.

Step D: Add the integrity constraints to the information structure.

In this step, the necessary integrity constraints were added to the information structure .

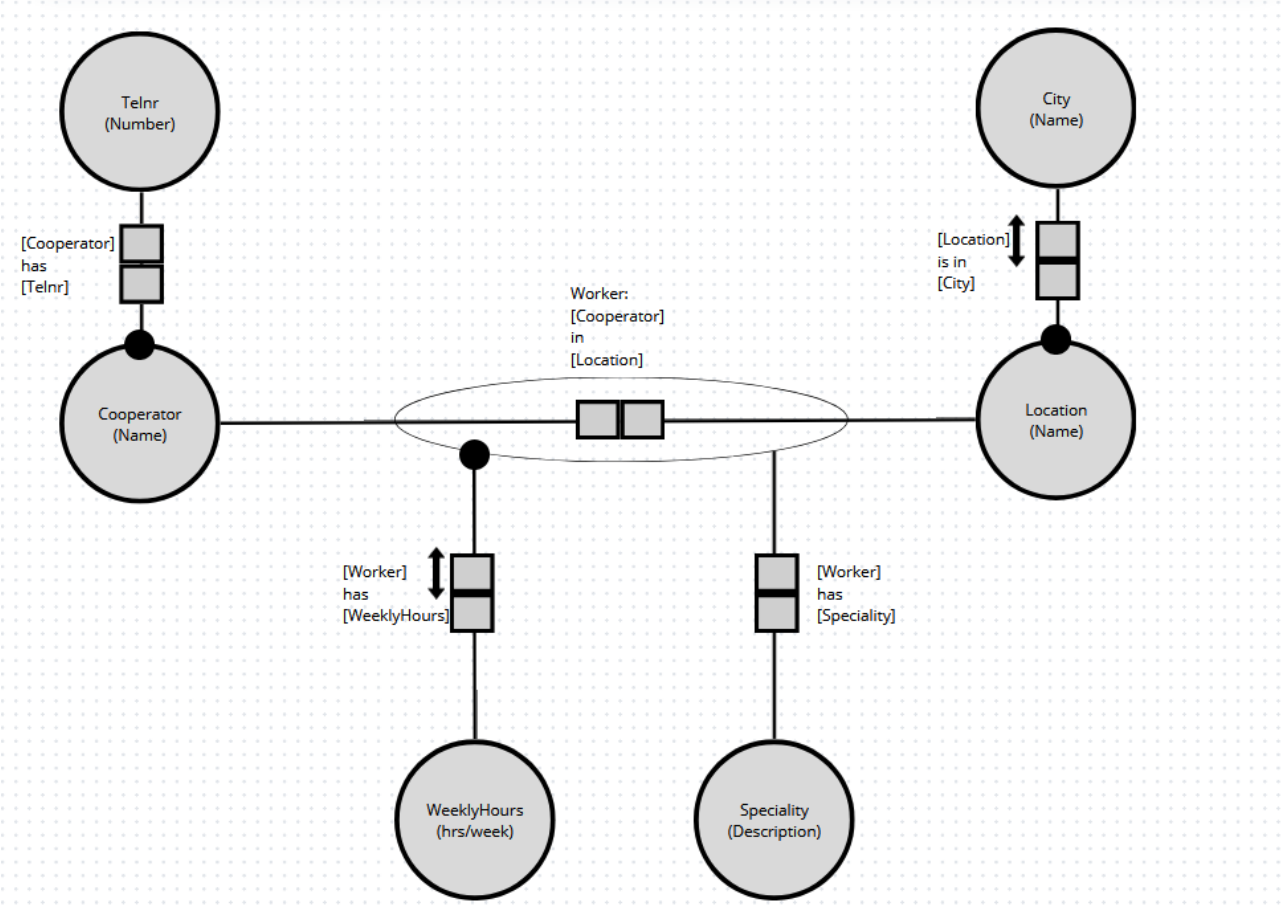


Diagram 3. Resulting Information Structure with Integrity Constraints.

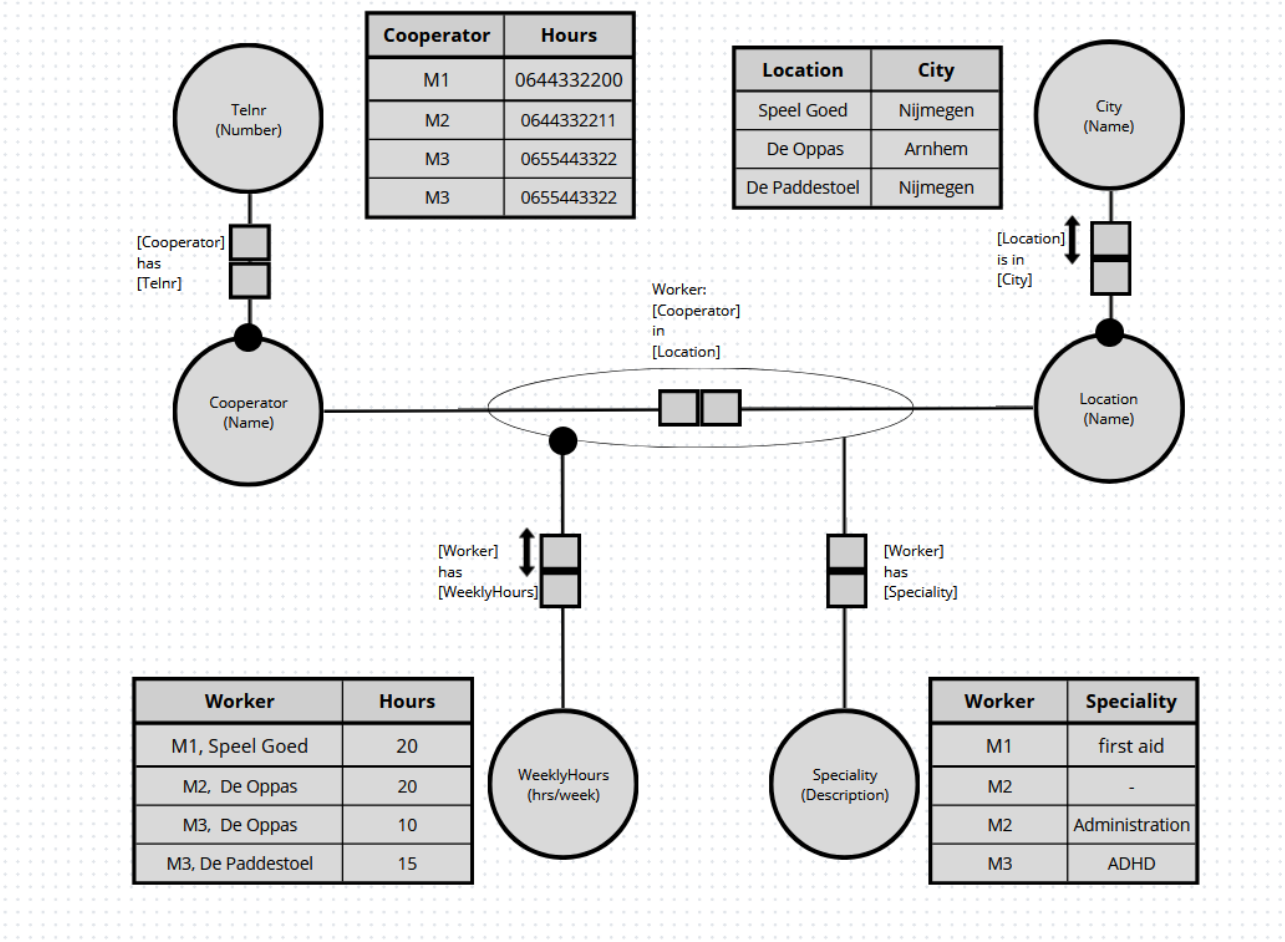
A Unique Rule Constarints was applied (by arrow) to ensure that value in a specific role cannot occur more than once. It guarantees that each instance is uniquely identifiable and prevents accidental duplicates. In our example, each *Location* must have a unique name. This is represented as a unique role in the schema. Any data instance that violates these uniqueness rules (e.g., two locations with the same name) would be rejected as invalid.

A Total Role Constraint means that all instances from associated object type must participate in a particular role. In other words, no object can exist without fulfilling this role. I added two contraints of that type: every *Worker* must have *WeeklyHours*. This ensures that there are no workers without any *WeeklyHours*. Every *Location* has a unique Name, which is also marked as a total role because every location must be identifiable.

Step E: place the original data instances (the population of the original table) inside the types of the information structure and check that your constraints are correct.

Finally, the original data instances from the given table were placed inside the types of the information structure. This step is referred to as the population of the schema.

This confirmed that the information structure, together with the applied integrity constraints, is correctly modelled and matches the original dataset.



Cooperator	Hours
M1	0644332200
M2	0644332211
M3	0655443322
M3	0655443322

Location	City
Speel Goed	Nijmegen
De Oppas	Arnhem
De Paddestoel	Nijmegen

Worker	Hours
M1, Speel Goed	20
M2, De Oppas	20
M3, De Oppas	10
M3, De Paddestoel	15

Worker	Speciality
M1	first aid
M2	-
M2	Administration
M3	ADHD

Diagram 4. Resulting Information Structure with Integrity Constraints.

Assignment 4

Additional Grammar Rules

In the assignment four, I was proposed with a new table (see Table 5 below) that gives an overview of the participants of an athletic contest, the country they represent and their birth country. The main goal of this assignment is to specify the usual Object Role Grammar Rules (ORGRs), but with specifying additional grammar rules. The work was done in four steps, where first three are already known from previous assignments:

- Step A: Give sentences in ORNF.
- Step B: Give the corresponding ORGR's.
- Step C: Give the resulting information structure with integrity constraints.
- Step D: Specify the additional grammar rules.

Athlete administration:

Athlete	Country	Birthplace
Ann Arbor	USA	USA
Bill Abbot	UK	?
Chris Lee	USA	NZ

Table 5. Athlete administration.

Step A: ORNF sentences.

1. **Athlete** with AName “Ann Arbor” represents **Country** with CName “USA”.
2. **Athlete** with AName “Bill Abbot” represents **Country** with CName “UK”.
3. **Athlete** with AName “Chris Lee” represents **Country** with CName “USA”.
4. **Athlete** with AName “Ann Arbor” was born in **Birthplace** with BName “USA”.
5. **Athlete** with AName “Bill Abbot” was born in **Birthplace** with BName “unknown”.
6. **Athlete** with AName “Chris Lee” was born in **Birthplace** with BName “NZ”.

Step B: ORGR.

1. **Athlete** (AName) represents **Country** (CName).
2. **Athlete** (AName) was born in **Birthplace** (BName).

Step C: Resulting Information Structure (ORM Schema) with integrity constraints.

Resulting Information Structure.

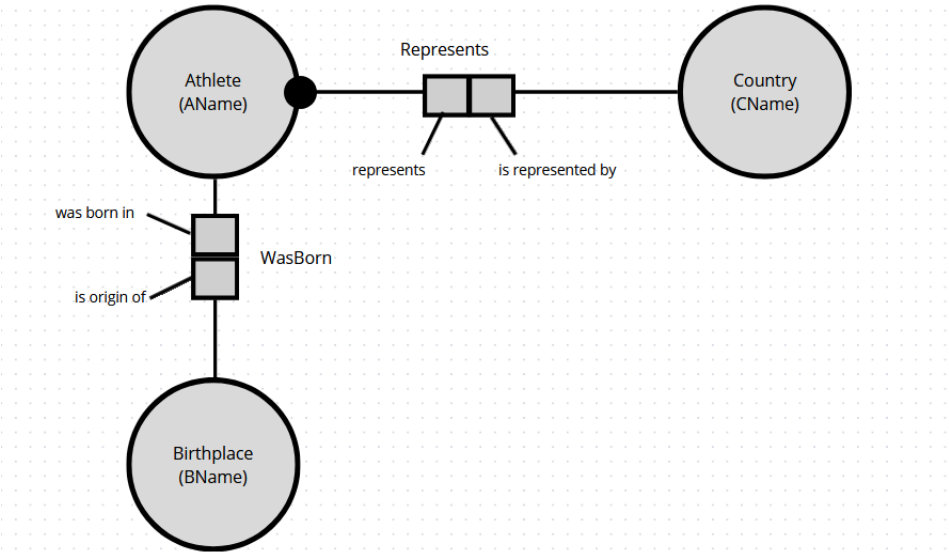


Diagram 5. Resulting Information Structure with Integrity Constraints.

Step D: Specify the additional grammar rules.

In previous assignments, when we used ORGR, we created an abstraction of sentences. The instances of every object have been removed, and the standard name is written on the sides, as shown below:

1. **Athlete** (AName) represents **Country** (CName).
2. **Athlete** (AName) was born in **Birthplace** (BName).

This is an abbreviated entry, a simple statement. It shows the basic idea of the rule, but does not detail how it works from all sides.

From this week, we need formal, strict and detailed description of the same rule. I will explain that on grammar rules of natural language, since it is similar to ORM/ORGR, with the main difference that natural language id for sentences structure description, and ORGR for fact types, object types structure description in the conceptual schema.

Formal grammar rule example: **sentence: subject, verb, object.**

Sentence, subject, verb, object are non-terminals, abstract syntactic categories, that can be divided into another elements. Terminals therefore, are the concrete language words, they cannot be divided into

smaller further in grammar bound (for example, “Anna”, “416”, “USA”, “represent”, “born in”, “has”, etc.).

As some words changes because of specification of particular word (e.g., “I walk”, “She walks”, “We walk” verb *walk* changes because of person and number), we need to consider context. For that we need more advanced constructors (parameters).

Parameters are added to syntactic category:

sentence(PERSON, NUMBER): subject(PERSON, NUMBER), verb(PERSON, NUMBER), object(PERSON, NUMBER).

Now, we need to possible add values for parameters (special rules):

PERSON :: FIRST | SECOND | THIRD
NUMBER :: SINGULAR | PLURAL .

All this together forms a two-level grammar, where we have rules for 1) sentence (structure), and 2) parameter values.

The new format is not a change of rules, but their formalization and interpretation. It turns a short phrase into a strict schema where all roles, communication directions, and data types are spelled out.

In our week’s assignment, we have two binary fact types:

- Represents: **Athlete**, “represents”, **Country**;
  Country, “is represented by”, **Athlete**.
- WasBorn: **Athlete**, “was born in”, **Birthplace**;
  Birthplace, “is origin of”, **Athlete**.

Three object types:

- **Athlete:** “Athlete with”, AName.
- **Country**: “Country with”, CName.
- **Birthplace**: “Birthplace with”, BName.

And three value types:

- AName: “AName”, string.
- CName: “CName”, string.
- BName: “BName”, string.

Assignment 5

Creating Paths through Information Structures (Object Role Calculus)

In the fifth assignment, the main task is to create paths through information structures by using the Object Role Calculus (ORC). Unlike the earlier weeks, where the focus was on constructing tables, writing ORNF sentences, giving corresponding ORGR’s, or specifying integrity constraints, this assignment focuses on how we can navigate through an information model in order to answer complex questions.

ORC is formal query language used to navigate through information structures. While ORM schemas describe object and their roles in a domain, ORC allows us to ask questions about these schemas in a precise and logical way. It works by following path through object and roles in the schema and then

expressing conditions that these objects must satisfy. With ORC, we can use label values to point to specific instances, combine conditions with logical operators like AND ALSO or BUT NOT, and even apply statistical functions such as MAXIMUM, AVERAGE, SUM, or COUNT. In this way, ORC connects the conceptual schema with the ability to retrieve information, making it possible to reason about the data stored in the model.

The following questions were provided, together with the ORM schemas (see Diagrams 6 and 7 with questions below). Each of seven questions must be expressed in ORC. We want to have (binary or unary/ternary) elementary fact types. If we have a ternary fact type, we should check whether it can be split without losing information, for example in two binary fact types. If it can be split, we must give the smaller fact types. Fact types with four or more roles are probably incorrect. If we would be satisfied with such large fact types, we could also put all information in one very big fact type. That is not information modelling, that is just throwing everything together, which we don't want.

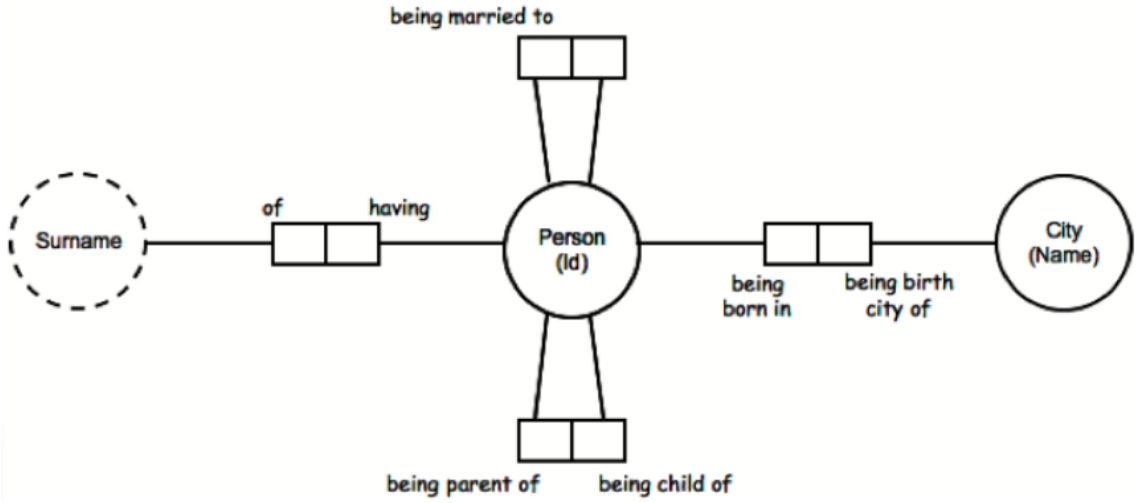


Diagram 6. Instruction week 5.1.

a) Give the surnames of all persons being married with someone born in Arnhem.
b) Give the surnames of all persons being married with someone born not in Arnhem

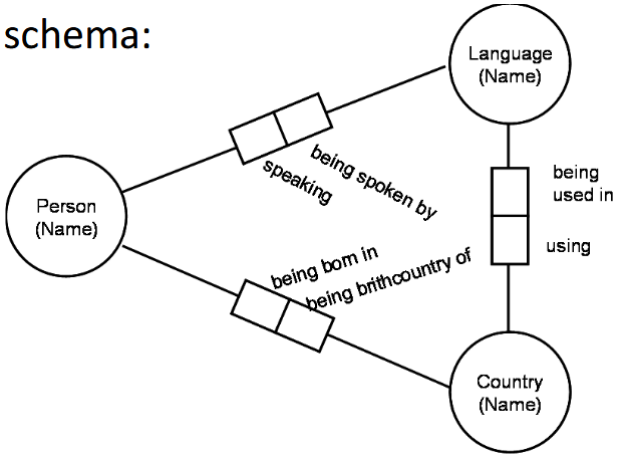


Diagram 7. Instruction week 5.2..

c) Which persons are speaking Greek?
d) Who is born in France and speaks German?
e) What languages are spoken in the country where Piet has been born?
f) What languages are spoken by both Piet and Els?
g) What languages are being spoken in a polyglot country?

a. Give the surnames of all persons being married with someone born in Arnhem.

The persons we are looking for must satisfy two conditions:

1. They have a surname
2. They are married with another person who has born in the city Arnhem.

This is expressed as the concatenation of the constructs ot-rn-ot-rn-ot- rn-ot-rn-ot-lv from the conceptual schema:

Surname *of* Person *being married to* Person *being born in* City *with* Name "Arnhem".

Structure: ot -rn - ot-rn-ot- rn-ot-rn-ot-lv

There ot means Object Type, rn Role Name, lv Label value. All role names are in italic in the ORC.

b. Give the surnames of all persons being married with someone born not in Arnhem.

This is similar to previous but now we must exclude persons born in Arnhem. For this we use operator BUT NOT to express set differences.

Surname *of* Person *being married to* Person *being born in* City
BUT NOT
Person *being born in* City *with* Name "Arnhem".

Structure: ot-rn-ot-rn-ot-rn-ot BUT NOT ot-rn -ot-rn-ot-lv

There we are looking for path with all surnames of persons being married with persons being born in city and subtract path with people who are not being born in Arnhem.

c. Which persons are speaking Greek?

ORNF: Person speaking Language with Name "Greek".

Name *of* Person *speaking* Language *with* Name "Greek".

Structure: ot-rn-ot-rn-ot -rn-ot-lv

The query selects all people who are connected through the role speaking to the language Greek.

d. Who is born in France and speaks German?

The persons must satisfy two conditions:

1. They are born in the country France.
2. They speak the language German.

We combine these conditions with the operator AND ALSO (intersection):

Name *of* Person *being born in* Country *with* Name "France"
AND ALSO
speaking Language *with* Name "German".

Structure: ot-rn-ot-rn-ot-rn-ot-lv AND ALSO (ot)-rn-ot-rn-ot-lv.

Ot is in scopes, because we can not mention ot twice as simplification of :

Name *of* Person *being born in* Country *with* Name "France"
AND ALSO
Person *speaking* Language *with* Name "German".

e. What languages are spoken in the country where Piet has been born?

1) Identify the country where the person Piet was born.

2) Extract the languages spoken in that country.

Name *of* Language *being used in* Country
FROM
Person *with* Name “Piet” *being born in* THAT Country .
Structure: ot-rn-ot-rn-ot FROM ot-rn-ot-lv-rn THAT ot.

There FROM is used to give the place where bring specific information. And THAT means correlation between two instances.

f. What languages are spoken by both Piet and Els?

We want to find languages that are common to both persons. The intersection is expressed with AND ALSO.

Name *of* Language *being spoken by* Country Person *with* Name “Piet”
AND ALSO
being spoken by Person *with* Name “Els”.
Structure: ot-rn-ot-rn-ot—rn-ot-lv AND ALSO (ot)-rn-ot-rn-ot-lv.

g. What languages are being spoken in a polyglot country?

We must select all languages, but only from those countries that are marked as polyglot in the schema. (Polyglotism is the ability to master multiple languages [wikipedia].)

Name *of* Language *being used in* Country *using* ANOTHER Name *of* Language.

Structure: ot-rn-ot-rn-ot-rn-ot.

Assignment 6

Specialization and Generalization

In the sixth assignment, I was provided with table (see Table 6 below) that provides information on participation and results of students from some educational institution.

- Some courses consist of a number of parts as in the case of R&D1 and R&D2. Such courses are referred to as a course lines.
- For each course or course part, the name of the teacher and his/her phone number is recorded in the table.
- Each participation year of a student to a course (part) is recorded in the table, and, if available, the result.

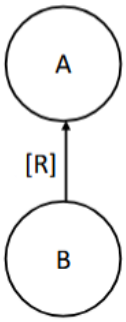
The main goal of this week was to make an information structure which contains specialization and generalization. These two concepts describe how different object types can be related to each other in a hierarchical way, based on shared or distinct characteristics. The main idea is that we will not duplicate the same data, logically combining them into one group and will be able to clearly reflect their relationships for simplified work with the model.

In order to complete this assignment, we need to give the necessary ORGR's for specialization and generalization and the resulting ORM schema, but we do not need to give the integrity constraints. In this week, it is not required to give the sentences in Object Role Normal Form (ORNF).

StudentNr	Studie	Course(line)	Part	Teacher	TelNr	Year	Mark
S091234	IK	R&D	1	Swinkels	51234	2007-2008	8
S092345	IC	R&D	2	Bergsma	53421	2006-2007	3
S092345	IC	R&D	2	Pietersen	54433	2007-2008	
S105432	IC	R&D	2	Pietersen	4433	2007-2008	7
S106543	IC	DM		Boomsma	53421	2007-2008	9

Table 6. Information about students, courses, and participation.

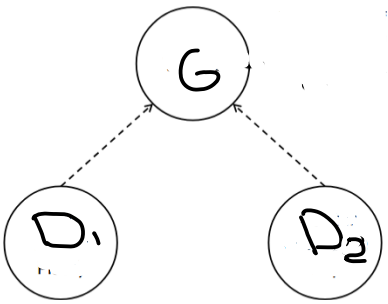
Specialization focuses on the *differences*, and is used to split one type into *subtypes*. It is the division of one object type (*supertype*) into several *subtypes* that inherit all characteristics from the supertype but also add specific attributes or roles that apply only to them. This is called homogeneous object types , when all subtypes come from the same source. It is often used when you want to refine or specify one general type.



In the schema is shown by arrow from B (it is the specialization of A) to A. [R] means subtype defining rule for B. Then population B can be derived by R: Pop(B) = {x∈Pop(A) | R(x)}. Subtype inherits all roles from its parent but adds its own specific ones.

For instance, *Course* can have subtypes *R&D1* and *R&D2* (the same course line, but different parts). From an objectified unary fact (one general fact about an object) we go to specializations (different detailed types).

Generalization, on the other hand, is the opposite process, it focuses on the *similarities*, and is used to merge different types into one. It is the process of identifying what is common among multiple different object types and combining into one general *supertype* that captures their shared properties. This represents heterogeneous object types, when merging different things under one concept.



In the schema is shown by dotted arrow from D1 and D2 to G. Generalization is used when different types play similar roles, and helps to avoid schema redundancy. If G is the generalization of D1 ... Dk we have: Pop (G) = $U_{1 \le x \le k}$ Pop (Dx). We call D1 ... Dk the specifiers of G.

For instance, subtypes *R&D1* and *R&D2* are generalized into *R&D* (a general course line that includes different part). Type hierarchy with heterogeneity, where distinct object types are grouped into a single overarching category.

Both specialization and generalization make an information model more structured and flexible. They help to avoid data duplication and allow us to represent inheritance, similar to object-oriented programming. In ORM (Object Role Modelling), this is represented by subtype-supertype relationships that show which object types share roles and which extend them.

Step A: ORGR.

1. **Student (**StudentNr**)** follows **Study (**StudyName**)**.

2. Participation: **Student (**StudentNr**)** participates in **Course (**CourseName**)** during **AcademicYear (**Year**)**.

3. Participation **(**Student, Course, AcademicYear**)** may have **Mark (**Score**)**.

4. **Teacher (**TeacherName**)** has **TelNr (**Number**)**.
5. **Teacher (**TeacherName**)** teaches **Course (**CourseName**)**.

6. Generalization:
   - Course GENERALIZES RegularCourse, CourseLine.

7. **CourseLine (**CourseLineName**)** IS **Course** with **Type "**Line".
8. **CoursePart (**PartName**)** IS part of **CourseLine** with **PartNumber** "N".
9. Specialization of Course:
   - **RegularCourse (**CourseRegularName**)** IS Course BUT NOT **CoursePart (**PartName**)**.
   - **CourseLine (**CourseLineName**)** IS Course with **CoursePart (**PartName**)**.
10. **CourseLine (**CourseLineName**)** consists of **CoursePart (**PartName**)**.

Step B : Resulting Information Structure (ORM Schema).

There we have Course as supertype with 2 subtypes (specializations) such as RegularCourse and CourseLine. CourseLine can be specified by CoursePart number.

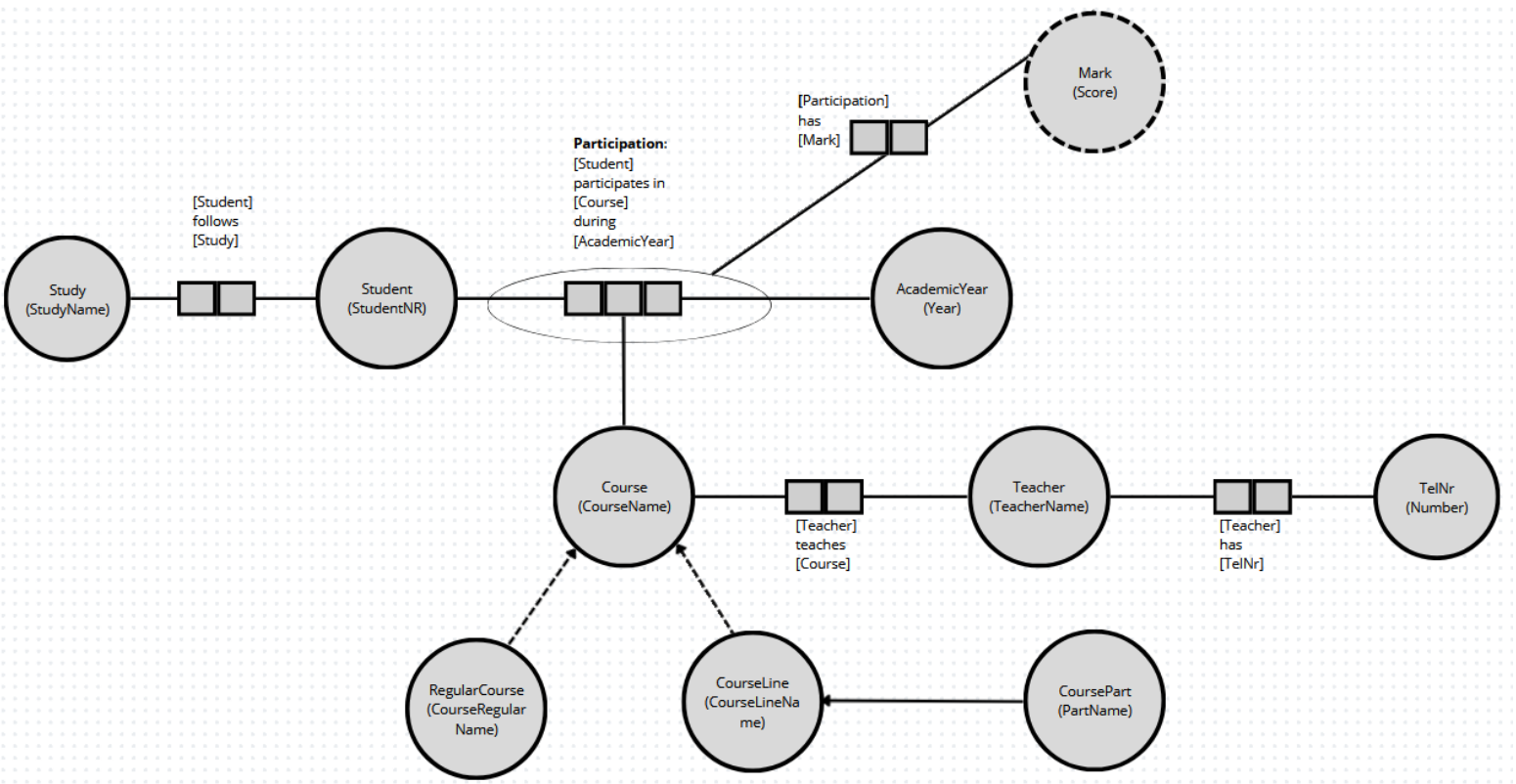


Diagram 7. Resulting Information Structure with specialization and generalization.

Assignment 7

In the last week's assignment of Part A, the exercises 7.1 - 7.4 were provided along with the answers (see Figures 1-7 below), which need to be clarified.

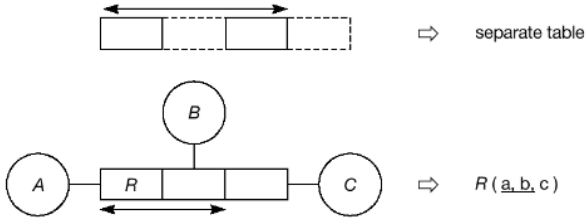
Main focus of this week is on model transformations. Model transformation is the process of converting one information model into another while preserving the meaning of the data. This allows us to reorganize or optimize models for different purposes, such as improving clarity, reducing redundancy, or preparing for database implementation.

Mapping procedure is also called grouping, because fact types are grouped. We can automatically avoid redundancy in tables by ensuring that each fact type maps to only one table, in such a way that its instances appear only once.

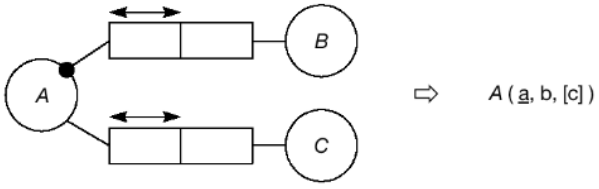
There are two basic rules:

1. Fact types with compound uniqueness constraints map to separate tables.
2. Fact types with functional roles attached to the same object type are grouped into the same table. The object type will be the key (uniqueness)

Compound uniqueness constraints:



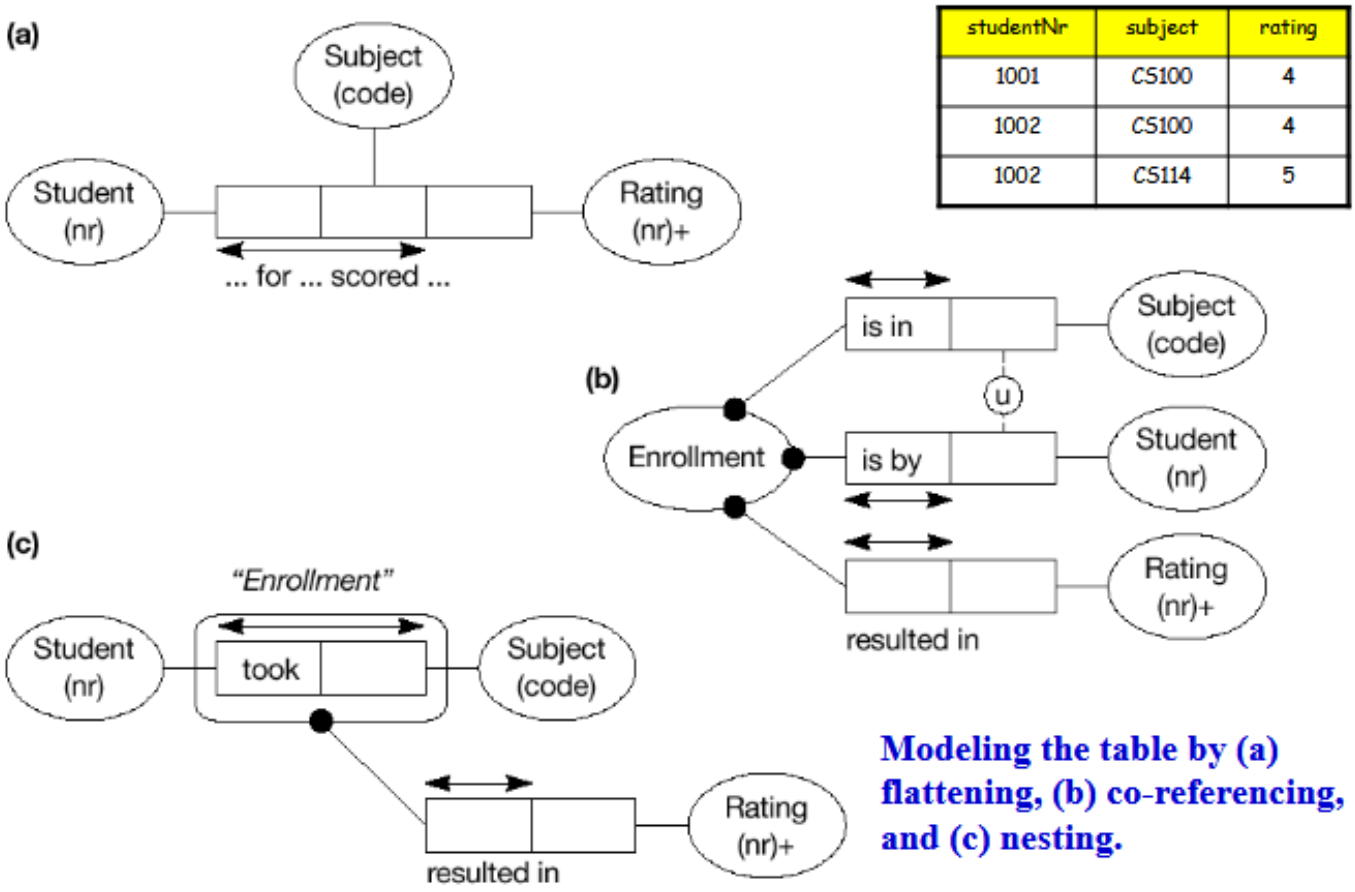
Functional roles attached to the same object type:



A key concept this week is equivalent schema transformations – to create an alternative representation of the same information. During pre-processing, a conceptual schema can be transformed into an equivalent schema that may be better suited for later steps, such as mapping to tables. This optimization can reduce the number of tables, simplify queries, and improve the clarity of the database design. However, it may also increase diagram complexity or require additional reasoning to maintain consistency.

Finally, advanced structuring techniques like nesting**,** co-referencing, and flattening are applied to control how objects and relationships are represented:

- Nesting embeds objects within others.
- Co-referencing links repeated entities across the schema.
- Flattening simplifies nested structures into flat tables for easier implementation.



studentNr	subject	rating
1001	CS100	4
1002	CS100	4
1002	CS114	5

Modeling the table by (a) flattening, (b) co-referencing, and (c) nesting.

These exercises build on previous skills in ORNF, ORGR, and ORM schemas, showing how structured information can be transformed, optimized, and prepared for implementation in different ways without losing meaning.

Instruction week 7

In this week you will get exercises 7.1 – 7.4.

We will give you the exercises and also the answers. In this week's assignment, you are requested to do the following:

- Examine the exercises and the answers.
- Write in your Project Report about these answers. To make this more concrete: you must write at least one whole page, in which you explain in your own words what is happening in 7.1, 7.2, 7.3 and 7.4.

It is not a big problem if there are details that you do not understand yet. But you must try to write a meaningful text to explain what is happening in the above four steps.

7.1

In the shown table you find the result of a wrestle competition between countries. In the preliminaries of this competition in each weight category many country teams are wrestling against others.

The best four teams in each category go on to the finals. Only the finalists are recorded, so not the participating countries that were already eliminated.

Each country may only delegate one team per category, so that in each category there will always be four different countries participating in the final round. In the finals they are competing for the first and the second position, indicated as Gold and Silver. In this competition there is no Bronze. It is not possible to have shared positions. In each category there is exactly one Gold and one Silver winner. There are 4 weight categories: Fly, Light, Middle and Heavyweight.

Weight category	Finalists	Gold	Silver
Flyweight	Uruguay	Belgium	Norway
	Belgium		
	Zambia		
	Norway		
Lightweight	Belgium	Belgium	Uruguay
	Uruguay		
	China		
	Mali		
Middleweight	Norway	Australia	China
	Canada		
	China		
	Australia		
Heavyweight	Uruguay	Japan	Belgium
	Japan		
	Netherlands		
	Belgium		

7.1

Make an ORM conceptual schema, including all constraints. Make use of the following not-elementary wording, that delivers three binary fact types:

"In the weight category flyweight the finalists were Belgium, Uruguay, Zambia and Norway. In lightweight it were Belgium, Mali, Uruguay and China. In lightweight Belgium won Gold and in middleweight Australia. In the category flyweight Norway won Silver, in middleweight China and in the category heavyweight Belgium".

7.2 – 7.4

7.2. Apply relational mapping to the obtained conceptual schema and give the resulting relational scheme. Indicate in the relational scheme all necessary constraints.

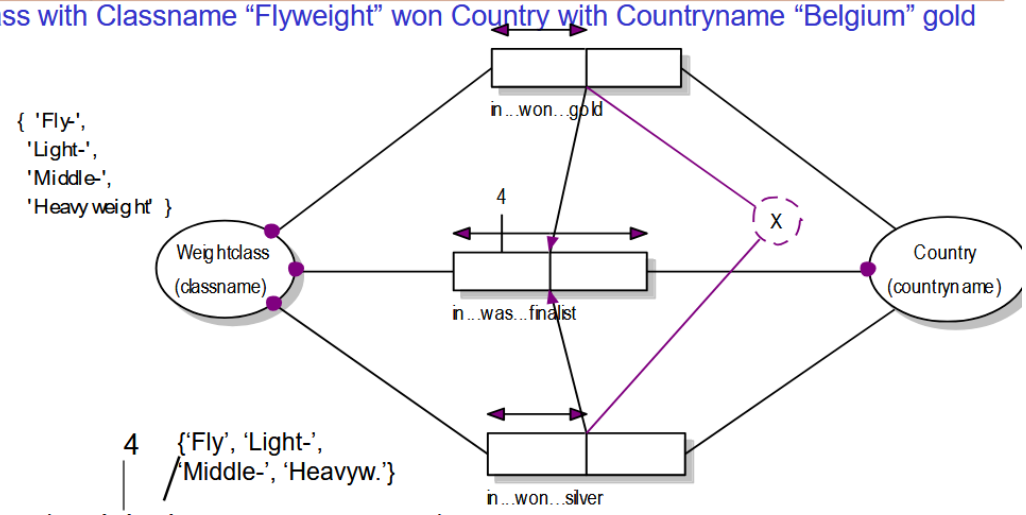
7.3. Apply a transformation to the conceptual schema of exercise 7.1 resulting in an equivalent conceptual schema with one binary and one ternary fact type. Adapt all existing constraints.

7.4. Now apply relational mapping to the new conceptual schema (produced in 7.3) and give the resulting relational scheme. Indicate in the relational scheme all necessary constraints.

Possible solution for exercise 7.1 and 7.2 (➔ 3 binary facttypes)

Elementary fact expression:

in Weightclass with Classname “Flyweight” won Country with Countryname “Belgium” gold



Mapping:

4 {‘Fly’, ‘Light-’, ‘Middle-’, ‘Heavyw.’}

Finalists (weightclass, countryname)

≠

MedalWinners (weightclass, country_with_gold, country_with_silver)

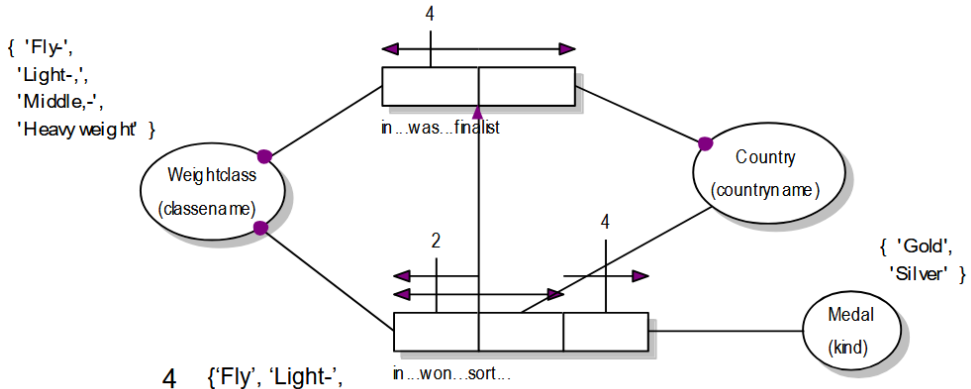
{‘Fly-’, ‘Light-’, ‘Middle-’, ‘Heavyweight’}

Possible solution for exercise 7.3 and 7.4

Transformation to one binary and one ternary fact type.

Elementary fact expression:

in Weightclass with Classname “Flyweight” won Country with Countryname “Belgium” sort Medal of Kind “Gold”



Mapping:

4 {‘Fly’, ‘Light-’, ‘Middle-’, ‘Heavyw.’}

Finalists (weightclass, countryname)

2 4 {‘Silver’, ‘Gold’}

Medalwinners (weightclass, countryname, medalkind)

{‘Fly’, ‘Light-’, ‘Middle-’, ‘Heavyw.’}

Figures 1,2,3,3,4,5,6 and 7. Week’s 7 Instructions

7.1) ORM

The main goal of this exercise is to build a conceptual ORM schema that formally describes the data from the given table. These schemas are later used for database implementation. In this case, the table describes the world of competitions (UoD), where there are countries, weight classes, and their results.

Two object types are identified, each with multiple instances:

1. Weightclass(classname): { 'Flyweight', 'Lightweight', 'Middleweight', 'Heavyweight' }
2. Country(countryname)

Three binary fact types describe real-world relationships between these objects:

1. Finalist(Weightclass, Country): «In the weight category … the finalists were 4 countries ...»
2. WonGold(Weightclass, Country): «In the weight category … 1 country … won Gold»
3. WonSilver(Weightclass, Country): «In the weight category … 1 country … won Silver»

To ensure correctness and avoid contradictions, several constraints are applied:

- Frequency constraint: There must be exactly 4 finalists in each category (Weightclass).
- Total role constraint: Each Weightclass category can have only one country with Gold and one with Silver.
- Total role constraint: In one Weightclass the same country can be a finalist at most once.
- Exclusion constraint: Countries winning Gold and Silver must be different (Gold ≠ Silver) within one Weightclass category.
- Subset constraint: Any country winning Gold or Silver must also be a finalist in the same Weightclass.
- Uniqueness constraint: Each weight class appears in all three steps of the competition; countries must appear in Finalist.

These constraints formalize the rules of the competition: no two countries can share Gold, and no country can win a medal without being a finalist.

7.2**) R**elational mapping

After creating the conceptual ORM schema, it must be converted into a relational schema, i.e., tables with columns and relationships. Each entity type and fact type from ORM becomes a table with clearly defined keys and constraints:

- Each entity → a separate table.
- Entity attributes (e.g., ID, Name) → table columns.
- Relationships → foreign keys to maintain data integrity.
- Constraints ensure correctness:
    - PRIMARY KEY: uniquely identifies each row.
    - FOREIGN KEY: ensures referenced values exist.
    - NOT NULL: column must have a value.
    - UNIQUE: column values must be unique.

For our three binary fact types, each relationship becomes a table:

- Finalists(weightclass, countryname)
- MedalWinners(weightclass, country_with_gold, country_with_silver)

Key constraints include:

- Uniqueness: No duplicate countries in a Weightclass.
- Only one Gold and one Silver per Weightclass category; they must be different countries.

Here, Weightclass is the primary key in Finalists. In MedalWinners, the primary key is weightclass, and the table lists the Gold and Silver winners. This relational model mirrors the ORM schema in table form.

Generalization of MedalWinners (weightclass, country_with_gold) and (weightclass, country_with_silver) to Finalists (weightclass, countryname) was shown with dashed arrows.

7.3**) Transformation to ORM resulting in an equivalent ORM with one binary and one ternary fact type**

Now the task requires simplifying the structure and making a scheme with one binary fact type (finalists), one ternary fact type (winners and type of medal).

1. The two binary relationships “Gold” и “Silver” are combined into one ternary relationship “MedalWinners”;
2. A new object type Medal is added, with an attribute kind (e.g., Gold, Silver).

The ternary fact type now reads: “In Weightclass … country … won medal of kind …”

This process is called flattening, which:

- Simplifies the data structure.
- Makes the schema more flexible (e.g., adding Bronze without a new table).
- Improves normalization and reduces redundancy.

The position of the constraints has been slightly changed, for example, the arrow from a ternary fact to a binary means: for each Weightclass, there can only be two countries winning medals, and each medal kind is counted correctly. Now there is a limit of two countries for each Weightclass in the medals.

7.4**) R**elational mapping

After the transformation in 7.3, the new relational schema is created:

- Finalists(weightclass, countryname) remains unchanged to fix all participants of Final.
- MedalWinners(weightclass, countryname, medalkind) replaces the previous table with Gold and Silver columns (MedalWinners (weightclass, country_with_gold, country_with_silver)):
    - Primary key: combination of weightclass and countryname.
    - medalkind values: ‘Gold’ or ‘Silver’.

Constraints:

- No duplicate medalkind in one Weightclass category.

- Each country can receive at most one medal per category.

The new relational schema is equivalent to the original but simpler and better structured.

Overall, this exercise reinforces how ORM schemas are built, how different schemas can represent the same data in multiple ways, and how choosing the right type of schema depends on the task.